

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.17

ROW LOCKING

 Copy as Markdown

Row locking

Row-level locking allows you to lock specific rows in the database to prevent other transactions from modifying them while you work. This is essential for safely handling concurrent updates, such as processing payments, managing inventory, or any scenario where two transactions might conflict.

WARNING

All row locking operations require a [transaction](#). An exception will be thrown if you attempt to acquire a lock without one.

For the following examples we will use this model:

```
class: Company
table: company
fields:
  name: String
```

Locking rows with a read

You can lock rows as part of a read operation by passing the `LockMode` parameter to `find`, `findFirstRow`, or `findById`. The locked rows are returned and held until the transaction completes.

```
await session.db.transaction((transaction) async {
  var companies = await Company.db.find(
    session,
    where: (t) => t.name.equals('Serverpod'),
    lockMode: LockMode.forUpdate,
    transaction: transaction,
  );

  // Rows are locked. It is safe to update without conflicts.
```



```

for (var company in companies) {
    company.name = 'Updated name';
    await Company.db.updateRow(session, company, transaction: transaction);
}
});

```

When a row is locked, other transactions that attempt to acquire a conflicting lock on the same rows will wait until the lock is released. Regular reads without a `LockMode` are not affected and can still read the rows freely. If waiting is not desired, you can configure the `lock behavior` to either throw an exception immediately or skip locked rows.

The `findFirstRow` and `findById` methods also support locking. Here's an example using `findById`:

```

await session.db.transaction((transaction) async {
    var company = await Company.db.findById(
        session,
        companyId,
        lockMode: LockMode.forUpdate,
        transaction: transaction,
    );

    if (company != null) {
        company.name = 'Updated name';
        await Company.db.updateRow(session, company, transaction: transaction);
    }
});

```

Locking rows without fetching data

If you only need to lock rows without reading their data, use the `LockRows` method. This acquires locks with less overhead since no row data is transferred.

```

await session.db.transaction((transaction) async {
    await Company.db.LockRows(
        session,
        where: (t) => t.name.equals('Serverpod'),
        lockMode: LockMode.forUpdate,
        transaction: transaction,
    );

    // Rows are locked. Perform updates using other methods.
});

```

Lock modes

The `LockMode` parameter determines the type of lock acquired. Different lock modes allow varying levels of concurrent access.

Lock Mode	Constant	Description
For update	<code>LockMode.forUpdate</code>	Exclusive lock that blocks all other locks. Use when you intend to update or delete the locked rows.
For no key update	<code>LockMode.forNoKeyUpdate</code>	Exclusive lock that allows <code>forKeyShare</code> locks. Use when updating non-key columns only.
For share	<code>LockMode.forShare</code>	Shared lock that blocks exclusive locks but allows other shared locks. Use when you need to ensure rows don't change while reading.
For key share	<code>LockMode.forKeyShare</code>	Weakest lock that only blocks changes to key columns.

For a detailed explanation of how lock modes interact, see the [PostgreSQL documentation](#). On SQLite, row locking does nothing, since SQLite only supports one write transaction at a time.

Lock behavior

The `LockBehavior` parameter controls what happens when a requested row is already locked by another transaction. If not specified, the default behavior is to wait.

Behavior	Constant	Description
Wait	<code>LockBehavior.wait</code>	Wait until the lock becomes available. This is the default.
No wait	<code>LockBehavior.noWait</code>	Throw an exception immediately if any row is already locked.
Skip locked	<code>LockBehavior.skipLocked</code>	Skip rows that are currently locked and return only the unlocked rows.

```
await session.db.transaction((transaction) async {
  var companies = await Company.db.find(
    session,
    where: (t) => t.id < 100,
    lockMode: LockMode.forUpdate,
    lockBehavior: LockBehavior.skipLocked,
    transaction: transaction,
```

```
);  
  
    // Only unlocked rows are returned.  
});
```

 INFO

LockBehavior.skipLocked is particularly useful for implementing job queues or work distribution, where multiple workers can each grab unlocked rows without waiting on each other.